

CADCAMWISE · SÉRIE TÉCNICA

NXOpen CAM com C#

do journal ao produto

Um livro prático construído a partir do PATHNC AUTOMATION: como o NX carrega uma DLL, como transformar journals em automação de verdade, como montar um painel Windows Forms dentro do NX, e quais padrões de projeto sustentam um sistema com 200+ classes.

Diógenes · CADCAMWise

Setembro de 2026 · Edição 1

NX 2406 · NXOpen C# / .NET Framework · Windows Forms · SQL Server Express

Sumário

Parte I — Fundamentos NXOpen

1. O que é o NXOpen e como o NX executa o seu código
2. Preparando o projeto no Visual Studio
3. Do journal gravado ao código reutilizável
4. Sessão, peça, tags e o ciclo de vida de um builder
5. A API de CAM: setup, grupos, operações e parâmetros
6. Seleção interativa e geometria: SelectionManager, Block UI Styler e assemblies
7. Drafting automático

Parte II — Windows Forms dentro do NX

8. Windows Forms essencial
9. Anatomia do painel PATHNC
10. Criando uma página nova, passo a passo
11. Janelas secundárias, grids e imagens

Parte III — Arquitetura e padrões

12. C# que o projeto usa: classes, interfaces, genéricos, LINQ
13. Padrões de projeto no PATHNC
14. Organização, testes e deploy
15. Compêndio de erros e soluções
16. Referências

1. O que é o NXOpen e como o NX executa o seu código

O Siemens NX expõe sua funcionalidade por uma família de APIs. Para C#, duas camadas importam:

Camada	Namespace	Estilo	Quando usar
NXOpen gerenciado	NXOpen , NXOpen.CAM , NXOpen.Drawings , NXOpen.BlockStyler ...	Orientado a objetos: Part , Body , Operation , builders	Padrão. É o que o gravador de journal gera.
UF (User Function)	NXOpen.UF via UFSession	Procedural, baseado em Tag e arrays	Quando o gerenciado não expõe algo: parâmetros CAM (UF_PARAM_*), bounding box, mass props, bordas de vista

No PATHNC os dois convivem o tempo todo. A regra prática: comece no gerenciado; quando faltar método, procure no UF pela função C equivalente (UF_MODL_ask_bounding_box vira uf.Modl.AskBoundingBox).

1.1 Três formas de executar

Forma	Como carrega	Prós	Contras
Journal (.cs)	Tools → Journal → Play compila e roda o arquivo na hora	Zero configuração; ideal para experimentar	Um arquivo só; sem depuração; recompila a cada execução
DLL (Journal → Play na DLL)	NX carrega a DLL, chama Main , descarrega conforme GetUnloadOption	Projeto completo no Visual Studio, várias classes, recursos, banco	Precisa compilar antes; para depurar, anexar ao ugraf.exe
Menu/ribbon customizado	UGII_USER_DIR + arquivos .men / .btn apontando para a DLL	Experiência de produto: botão na barra do NX	Configuração de ambiente por máquina

O erro Unable to load DLL 'libugini' aparece quando você executa a DLL/EXE fora do NX (F5 no Visual Studio, ou clicando no .exe). O NXOpen só encontra as bibliotecas nativas dentro do processo do NX ou com o NXBIN no PATH (modo externo, que exige licença própria). O PATHNC roda sempre via Tools → Journal → Play.

1.2 O contrato mínimo: Main e GetUnloadOption

Toda DLL ou journal que o NX executa precisa de um ponto de entrada e de uma resposta sobre quando descarregar:

Esqueleto de qualquer journal/DLL

```
using NXOpen;

public class NXJournal
{
    public static void Main(string[] args)
    {
        Session theSession = Session.GetSession(); // a sessao do NX que esta rodando
        Part workPart = theSession.Parts.Work;      // a Work Part atual (pode ser null)
        // ... seu codigo
    }

    // Immediately: descarrega ao terminar (journals simples)
    // Explicitly: fica carregada ate File -> Unload Shared Image (forms modeless!)
    public static int GetUnloadOption(string dummy)
    {
        return (int)Session.LibraryUnloadOption.Immediately;
    }
}
```

O PATHNC devolve Explicitly no Form1 , porque o painel é uma janela modeless: se a DLL fosse descarregada ao sair do Main , os botões deixariam de responder. Isso vai voltar no capítulo 8.

2. Preparando o projeto no Visual Studio

2.1 Tipo de projeto e referências

1. **Class Library (.NET Framework)** — não .NET Core/.NET 6+. O NX 2406 hospeda o runtime do .NET Framework 4.8. O PATHNC nasceu como Windows Forms App e é executado como DLL; funciona, mas Class Library é o tipo "certo".
2. **Plataforma x64** obrigatória; o NX é 64 bits. Um projeto *AnyCPU* com *Prefer 32-bit* marcado falha ao carregar.
3. **Referências** de `C:\Program Files\Siemens\NX2406\NXBIN\managed\`: `NXOpen.dll`, `NXOpen.UF.dll`, `NXOpenUI.dll`, `NXOpen.Utilities.dll`. Em todas: **Copy Local = False** — o NX já tem essas DLLs em memória; copiar gera conflito de versão.
4. Para Windows Forms: `System.Windows.Forms`, `System.Drawing`. Para SQL: `System.Data` (já vem no framework).

2.2 Recursos que viajam com a DLL

Dois tipos de arquivo precisam estar ao lado da DLL compilada. O caminho é resolvido em tempo de execução a partir da própria DLL — nunca fixo em `D:\...`:

```
ROUGHINTERACTIVE / ShopDocRepository - caminhos relativos à DLL

// Block UI Styler: o .dlx fica em Resources\ ao lado da DLL
string assemblyDir = System.IO.Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
theDlxFileName = System.IO.Path.Combine(assemblyDir, "Resources", "CAVITY_MILL.dlx");
if (!File.Exists(theDlxFileName))
    throw new FileNotFoundException("Arquivo .dlx nao encontrado em: " + theDlxFileName);

// Connection string: arquivo de texto ao lado da DLL (ver capítulo 14)
string arquivo = System.IO.Path.Combine(assemblyDir, "PathNCAutomationDB.connection");
```

No Visual Studio, marque esses arquivos com **Build Action = Content** e **Copy to Output Directory = Copy if newer**. Imagens usadas nos cards (`Resources.Screenshot_...`) vão para `Properties\Resources.resx` e são embutidas na DLL — não precisam de arquivo externo.

2.3 Depurar dentro do NX

1. Compile em **Debug**.
2. Abra o NX e carregue a DLL uma vez (Journal → Play).
3. No Visual Studio: Debug → Attach to Process → `ugraf.exe` (marque *Managed (.NET Framework)*).
4. Coloque breakpoints; execute a ação no NX.

Recompilar com o NX aberto: com `GetUnloadOption = Explicitly`, a DLL fica travada e o build falha com *file in use*. Use File → Utilities → Unload Shared Image no NX antes de compilar de novo, ou devolva `Immediately` enquanto estiver desenvolvendo journals sem janela.

2.4 Estrutura de pastas do PATHNC

```
Organização por módulo (um namespace por pasta)

nxteste2/
├─ Form1.cs                painel principal (partial class, ~1600 linhas)
├─ Form1.Designer.cs       gerado pelo VS - so InitializeComponent()
├─ Theme.cs                paleta, botoes, cartoes (capítulo 9)
├─ AI_COPILOT_MODELS/      StrategyHistoryService, StrategyApplier, CopilotResultForm
├─ SHOP_DOCUMENTATION/    SHOP_DOCUMENTATION_PT/EN/DE/..., ShopDocDatabase.cs
├─ QUOTE/                  QuoteService, QuoteForms
├─ FBM_MACHINING/          Strategies/ (IMachiningStrategy, MachiningContext)
├─ FBM_DRILLINGS/          AUTODRILL_GEOMETRIA_PROPRIA, MASTER_DRILLS, ...
├─ BLOCK_STYLER/           ROUGHINTERACTIVE + CAVITY_MILL.dlx
├─ WORKPLANES/             TOP_VIEW, FRONT_VIEW, ... (NX_3_PLUS_TWO_WCS)
├─ TEMPLATES_3+2/          THREE_PLUS_TWO_ROUGH/REST_MILL/SEMI_FINISH/PIPELINE
├─ TOOL_LIBRARY_FROM_NX/   ToolLibraryFromNX (224 LoadTool)
├─ Utilities/              HoleDiameterColorizer, ImportViseComponent, ApiKeyStore
└─ Resources/              CAVITY_MILL.dlx, imagens dos cards
```

3. Do journal gravado ao código reutilizável

O gravador de journal (Tools → Journal → Record) é o melhor professor de NXOpen que existe — e o pior programador. Ele registra *tudo* que a interface fez, inclusive o que não importa. Aprender NXOpen é aprender a ler um journal e separar o essencial do ruído.

3.1 O que o gravador produz

Quando você abre uma operação Cavity Mill e seleciona uma Trim Boundary, o journal tem 600 linhas. Estas são as categorias:

Categoria	Exemplo	Manter?
Navegação de câmera	<code>WorkView.ZoomAboutPoint(...), SetRotationTranslationScale(...)</code>	Não — rastro do mouse
Undo marks do diálogo	<code>SetUndoMark(Invisible, "Start"), DeleteUndoMark(...)</code>	Não — substitua por um único undo mark seu
Expressões/planos temporários	<code>CreateSystemExpressionWithUnits("0", unit1)</code> repetido 12 vezes, <code>plane1.DestroyPlane()</code>	Não — efeito colateral de abrir o diálogo
Referências a objetos por nome interno	<code>component1.FindObject("PROTO#.Features 3940D535102_REV_A(3) EDGE * 200222 * 200288 {...}")</code>	Não — só existe naquela peça
A sequência builder → propriedade → Commit	<code>CreateCavityMillingBuilder, TrimBoundary.BoundaryList, AppendCurves, Commit, Destroy</code>	Sim — é o núcleo

3.2 O núcleo extraído

Das 600 linhas, sobram estas. Note o que mudou: as arestas fixas viraram uma seleção do usuário, e a operação vem da ONT ou de uma constante.

● ● ● SELECIONAR_TRIM_BOUNDARY.cs – o essencial de 600 linhas gravadas

```
CavityMilling cav = (CavityMilling)workPart.CAMSetup.CAMOperationCollection.FindObject("CAVITY_MILL_TOP");
CavityMillingBuilder builder = workPart.CAMSetup.CAMOperationCollection.CreateCavityMillingBuilder(cav);

BoundarySetList setList = builder.TrimBoundary.BoundaryList;
BoundaryMillingSet bset = (BoundaryMillingSet)setList.FindItem(0);

NXOpen.Plane plane = workPart.Planes.CreatePlane(new Point3d(0,0,0), new Vector3d(0,0,1),
SmartObject.UpdateOption.AfterModeling);

bset.Plane = plane;
bset.ToolSide = BoundarySet.ToolSideTypes.OutsideOrRight;
bset.AppendCurves(curvesSelecionadasPeloUsuario, new Point3d(), identidade);

builder.Commit();
builder.Destroy();
```

3.3 A pergunta que não tem resposta no journal

"Dá para pausar o journal no diálogo e deixar o usuário selecionar?" Não. Ao reproduzir, o NX não abre o diálogo — só repete as chamadas. A interação precisa ser *sua*: `SelectionManager` (modal, simples) ou `Block UI Styler` (modeless, profissional). Os dois estão no capítulo 6.

3.4 Método de trabalho

1. Grave a ação manualmente, uma vez, na menor peça possível.
2. Apague câmera, undo marks intermediários, expressões e planos temporários.
3. Substitua cada `FindObject("nome fixo")` por uma busca (por nome de operação, por seleção do usuário, por varredura da coleção).
4. Envolve em `try/catch (NXException)` com `UndoToMark` no erro.
5. Extraia o que se repete para um método com parâmetros — é aqui que um journal vira uma classe do PATHNC.

Quando a API gerenciada não expõe algo — por exemplo, o RPM de uma operação não aparece como propriedade da `Operation` — o journal também não ajuda. A saída é a camada UF: `uf.Param.AskDoubleValue(op.Tag, UFConstants.UF_PARAM_SPINDLE_RPM, out rpm)`. Foi assim que o Shop Doc passou a ler RPM, avanço, diâmetro e número da ferramenta.

4. Sessão, peça, tags e o ciclo de vida de um builder

4.1 Os objetos que todo código começa pegando

Objetos de entrada

```
Session theSession = Session.Session();           // unica por processo (Singleton)
UI theUI = UI.UI();                               // seleção, NXMessageBox, dialogs
UFSession uf = UFSession.UFSession();             // camada UF
Part workPart = theSession.Parts.Work;            // a peça sendo editada
Part displayPart = theSession.Parts.Display;       // a peça mostrada (assembly raiz)
ListingWindow lw = theSession.ListingWindow;       // "console" do NX: lw.Open(); lw.WriteLine(...)
```

Work Part ≠ Display Part em assemblies. No setup CAM criado sobre uma peça importada, a Display Part é o assembly de manufatura e a peça original é um *componente*. Esse detalhe causou o bug de seleção de faces do Cavity Mill (capítulo 6.4).

4.2 Tag: o elo entre as duas camadas

Todo objeto do NX tem um `Tag` (um inteiro). O gerenciado esconde isso atrás de classes; o UF trabalha só com ele. A ponte é `objeto.Tag`:

Tag na prática

```
double[] box = new double[6];                    // xmin, ymin, zmin, xmax, ymax, zmax
uf.Modl.AskBoundingBox(body.Tag, box);           // UF recebe o Tag do Body gerenciado

double rpm;
uf.Param.AskDoubleValue(operation.Tag, UFConstants.UF_PARAM_SPINDLE_RPM, out rpm);
uf.Param.SetDoubleValue(operation.Tag, UFConstants.UF_PARAM_FEED_CUT, 1800.0);
```

4.3 Undo marks: desfazer é obrigatório

Um undo mark visível vira uma entrada no Ctrl+Z do usuário. No erro, `UndoToMark` devolve a peça ao estado anterior — e é isso que separa uma automação confiável de uma que "estraga a peça quando dá erro".

StrategyApplier.Apply – undo mark como transação do NX

```
Session.UndoMarkId mark = theSession.SetUndoMark(Session.MarkVisibility.Visible, "Copilot: apply strategy");
try
{
    // ... cria grupos, ferramentas, operacoes
}
catch
{
    theSession.UndoToMark(mark, null);           // tudo o que foi criado some
    throw;                                       // o chamador decide o que mostrar
}
```

4.4 Builder: Create → configurar → Commit → Destroy

Quase toda criação ou edição no NX segue este padrão. O builder é um objeto temporário que acumula parâmetros; `Commit` aplica e devolve o objeto final; `Destroy` libera o builder. **Sempre** destrua — inclusive no erro.

Padrão builder com finally

```
CavityMillingBuilder builder = null;
try
{
    builder = workPart.CAMSetup.CAMOperationCollection.CreateCavityMillingBuilder(operation);
    builder.CutPattern.CutPattern = CutPatternBuilder.Types.FollowPeriphery;
    builder.BndStepover.PercentToolFlatBuilder.Value = 50.0;
    builder.CutParameters.PartStock.Value = 0.5;
    NXObject result = builder.Commit();
}
finally
{
    if (builder != null) builder.Destroy();
}
```

4.5 Mensagens e diagnóstico

Ferramenta	Uso
<code>theUI.NXMessageBox.Show(titulo, DialogType.Error, texto)</code>	Erro que o usuário precisa ver (modal)
<code>lw.Open(); lw.WriteLine(...)</code>	Log detalhado; não interrompe
<code>MessageBox.Show(...)</code> (WinForms)	Dentro do painel; mesma coisa, estilo Windows
<code>ReportProgress(...)</code> (PATHNC)	Barra de status do painel — feedback contínuo (cap. 9)
<code>catch (NXException ex) { ex.ErrorCode; ex.Message; }</code>	Erros do NX carregam código numérico; <code>AssertErrorCode(1050029)</code> ignora um específico

5. A API de CAM: setup, grupos, operações e parâmetros

5.1 O modelo mental

Um setup CAM é uma árvore com quatro "vistas" (as abas do Operation Navigator). Cada operação pertence simultaneamente a um nó de cada vista:

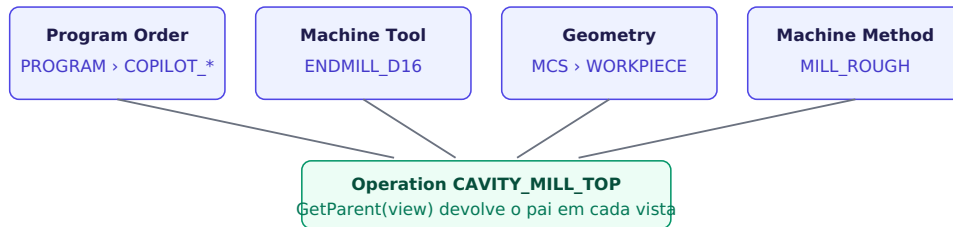


Figura 2 — Uma operação, quatro pais: as vistas do Operation Navigator.

```
ShopDocCollector – varrendo o setup

CAMSetup setup = workPart.CAMSetup;

foreach (NXOpen.CAM.Operation op in setup.CAMOperationCollection)
{
    string programa = op.GetParent(CAMSetup.View.ProgramOrder).Name;
    string ferramenta = op.GetParent(CAMSetup.View.MachineTool).Name;
    string geometria = op.GetParent(CAMSetup.View.Geometry).Name;
    string metodo = op.GetParent(CAMSetup.View.MachineMethod).Name;
    double minutos = op.GetToolpathTime();
    string tipo = op.GetType().Name; // "CavityMilling", "ZLevelMilling", "HoleDrilling"...
}
```

5.2 Encontrar grupos com segurança

FindObject("NONE") pode devolver um grupo de *outra* vista com o mesmo nome — e o cast falha. O PATHNC varre a coleção filtrando pelo tipo pedido:

```
ROUGHINTERACTIVE – busca genérica por nome + tipo

private static T FindGroupByNameAndType<T>(Part workPart, string groupName) where T : class
{
    foreach (CAMObject obj in workPart.CAMSetup.CAMGroupCollection.ToArray())
    {
        NCGroup group = obj as NCGroup;
        if (group == null || !string.Equals(group.Name, groupName, StringComparison.OrdinalIgnoreCase))
            continue;
        T match = obj as T; // Method, FeatureGeometry, NCGroup de programa...
        if (match != null) return match;
    }
    return null;
}

// uso:
Method method = FindGroupByNameAndType<Method>(workPart, "MILL_ROUGH");
FeatureGeometry workpiece = FindGroupByNameAndType<FeatureGeometry>(workPart, "WORKPIECE");
```

5.3 Criar uma operação do zero

```
CreateWithUserName

NXOpen.CAM.Operation op = setup.CAMOperationCollection.CreateWithUserName(
    programGroup, // pai em Program Order
    method, // pai em Machine Method
    tool, // pai em Machine Tool
    workpiece, // pai em Geometry
    "mill_contour", // template (arquivo mill_contour.opt em MACH\resource\template_set)
    "CAVITY_MILL", // subtipo dentro do template
    OperationCollection.UseDefaultName.False,
    "CAVITY_MILL_D16", // nome
    "CAVITY_MILL_D16");
```

Template e subtipo são texto e variam por instalação; se o NX reclamar, os nomes válidos estão nos arquivos .opt . O

StrategyApplier mantém um dicionário tipo → (template, subtipo) justamente para isso ser ajustável num só lugar.

5.4 Ferramentas e parâmetros via UF

StrategyApplier – ferramenta, parâmetros, geração

```
// criar fresa generica se nao existir no setup
NCGroup toolRoot = setup.GetRoot(CAMSetup.View.MachineTool);
NXOpen.CAM.Tool nova = (NXOpen.CAM.Tool)setup.CAMGroupCollection.CreateTool(
    toolRoot, "mill_planar", "MILL", NCGroupCollection.UseDefaultName.False, "ENDMILL_D16");
uf.Param.SetDoubleValue(nova.Tag, UFConstants.UF_PARAM_TL_DIAMETER, 16.0);
uf.Param.SetIntValue(nova.Tag, UFConstants.UF_PARAM_TL_NUMBER, 3);

// parametros de corte na operacao
uf.Param.SetDoubleValue(op.Tag, UFConstants.UF_PARAM_SPINDLE_RPM, 6000);
uf.Param.SetDoubleValue(op.Tag, UFConstants.UF_PARAM_FEED_CUT, 1800);

// gerar caminho
setup.GenerateToolPath(new CAMObject[] { op });
```

CreateTool devolve NCGroup, não Tool : o cast explícito é obrigatório. E a constante do avanço é UF_PARAM_FEED_CUT — não existe _VALUE .

5.5 Geometria de corte: ScCollector e regras

A Cut Area de uma operação é um *collector* que aceita *regras* (faces, corpos, features). A regra é criada pela ScRuleFactory da **peça dona das faces**:

ROUGHINTERACTIVE.ApplyCutAreaGeometry – o fix do assembly

```
builder.CutAreaGeometry.InitializeData(false);
GeometrySet geomSet = (GeometrySet)builder.CutAreaGeometry.GeometryList.FindItem(0);

// agrupa faces por OwningPart (assembly!) e cria uma regra por dono
foreach (var kvp in facesByOwner)
{
    Part ownerPart = kvp.Key;
    SelectionIntentRuleOptions opt = ownerPart.ScRuleFactory.CreateRuleOptions();
    opt.SetSelectedFromInactive(true);
    FaceDumbRule rule = ownerPart.ScRuleFactory.CreateRuleFaceDumb(kvp.Value.ToArray(), opt);
    opt.Dispose();
    rules.Add(rule);
}
geomSet.ScCollector.ReplaceRules(rules.ToArray(), false);
```

6. Seleção interativa e geometria

6.1 SelectionManager: a caixa de seleção modal

É a forma mais rápida de pedir algo ao usuário. A chamada bloqueia até OK/Cancel:

● ● ● SELECIONAR_TRIM_BOUNDARY – máscara de arestas e curvas

```
Selection.MaskTriple[] mask =
{
    // arestas de solido: tipo solid + subtipo 0 + flag ANY_EDGE (subtipo e flag juntos anulam o filtro)
    new Selection.MaskTriple(UFConstants.UF_solid_type, 0, UFConstants.UF_UI_SEL_FEATURE_ANY_EDGE),
    new Selection.MaskTriple(UFConstants.UF_line_type, 0, 0),
    new Selection.MaskTriple(UFConstants.UF_circle_type, 0, 0),
    new Selection.MaskTriple(UFConstants.UF_spline_type, 0, 0),
};
TaggedObject[] selected;
Selection.Response resp = theUI.SelectionManager.SelectTaggedObjects(
    "Selecione as curvas/arestas", "Trim Boundary",
    Selection.SelectionScope.AnyInAssembly, // pega componentes tambem
    Selection.SelectionAction.ClearAndEnableSpecific,
    false, false, mask, out selected);

if (resp != Selection.Response.Ok && resp != Selection.Response.ObjectSelected)
    return; // usuário cancelou
```

Quero selecionar	MaskTriple
Faces de sólido	(UF_solid_type, 0, UF_UI_SEL_FEATURE_ANY_FACE)
Arestas de sólido	(UF_solid_type, 0, UF_UI_SEL_FEATURE_ANY_EDGE)
Corpos inteiros	(UF_solid_type, 0, UF_UI_SEL_FEATURE_BODY)
Curvas soltas	(UF_line_type, 0, 0), (UF_circle_type, 0, 0) ...
Qualquer coisa (diagnóstico)	SelectionAction.AllAndDisableSpecific com array vazio

Diagnóstico que usamos: quando "não dá para selecionar nada", troque temporariamente para AllAndDisableSpecific sem máscara e imprima t.GetType().FullName de cada objeto selecionado na Listing Window. Se o clique pega o componente inteiro, a máscara está errada; se não pega nada, o problema é o contexto (componente lightweight, filtro global do NX).

6.2 Block UI Styler: diálogos como os do NX

O Block UI Styler (dentro do NX) desenha o diálogo num arquivo .dtx e gera a classe C# com os callbacks. É modeless — o usuário gira a peça enquanto seleciona. A classe gerada tem uma estrutura fixa:

```
ROUGHINTERACTIVE – esqueleto Block Styler

public class ROUGHINTERACTIVE
{
    private BlockDialog theDialog;
    private ListBox list_box0;           // Block type: List Box
    private IntegerBlock integerRpm;     // Block type: Integer
    private FaceCollector face_select0;  // Block type: Face Collector

    public ROUGHINTERACTIVE()
    {
        theDialog = theUI.CreateDialog(theDlxFileName);
        theDialog.AddInitializeHandler(new BlockDialog.Initialize(initialize_cb));
        theDialog.AddUpdateHandler(new BlockDialog.Update(update_cb));
        theDialog.AddOkHandler(new BlockDialog.Ok(ok_cb));
    }

    public void initialize_cb()           // roda antes de mostrar: popular listas, escopo
    {
        list_box0 = (ListBox)theDialog.TopBlock.FindBlock("list_box0");
        face_select0 = (FaceCollector)theDialog.TopBlock.FindBlock("face_select0");
        PopulateToolListBox();           // ferramentas CUTTER_/ENDMILL_ do setup
    }

    public int update_cb(UIBlock block) { return 0; } // reage a mudancas nos blocos

    public int ok_cb()                   // o trabalho de verdade
    {
        RunCavityMill();
        return 0;
    }
}
```

Propriedades dos blocos são acessadas por nome via `PropertyList`. Como os nomes mudam entre versões, o código do PATHNC tenta o nome validado e cai num fallback:

```
GetListBoxSelectedItems – leitura defensiva de PropertyList

PropertyList props = listBox.GetProperties();
try
{
    string[] result = props.GetStrings("SelectedItems");
    if (result != null && result.Length > 0) return result;
}
catch { /* nome nao existe nesta versao - fallback por indices */ }
int[] idx = props.GetIntegerVector("SelectedItems");
string[] all = props.GetStrings("ListItems");
```

6.3 Escopo: Work Part vs assembly

O `FaceCollector` nasce com escopo "Within Work Part Only". Quando o setup é um assembly e a peça é componente, nada se seleciona. A propriedade que governa isso é editada no próprio Block UI Styler (bloco → **Scope**), ou — se o bloco não expõe — o bloco é trocado por um `SelectObject`, que expõe `Scope` e aceita filtro por código:

```
SelectObject com filtro de faces

face_select0 = (SelectObject)theDialog.TopBlock.FindBlock("face_select0");
Selection.MaskTriple[] mask = {
    new Selection.MaskTriple(UFConstants.UF_solid_type, 0, UFConstants.UF_UI_SEL_FEATURE_ANY_FACE)
};
face_select0.SetSelectionFilter(Selection.SelectionAction.ClearAndEnableSpecific, mask);
```

6.4 Assemblies: componentes, protótipos e donos

Conceito	Código	Nota
Raiz do assembly	<code>workPart.ComponentAssembly.RootComponent</code>	<code>null</code> se a peça não é assembly
Filhos	<code>root.GetChildren()</code>	Array de <code>Component</code>
Peça de um componente	<code>(Part)component.Prototype</code>	É onde estão os <code>Bodies</code>
Dono de uma face selecionada	<code>face.OwningPart</code> as <code>Part</code>	Pode ser diferente da Work Part — regra da <code>ScRuleFactory</code> vem daqui
Corpos de toda a montagem	Ver <code>TodosOsCorpos()</code> abaixo	Usado pela assinatura geométrica do Copilot

ShopDocCollector.TodosOsCorpos – yield return

```
private static IEnumerable<Body> TodosOsCorpos(Part workPart)
{
    foreach (Body b in workPart.Bodies) yield return b;           // corpos da propria peca
    Component root = workPart.ComponentAssembly.RootComponent;
    if (root == null) yield break;
    foreach (Component c in root.GetChildren())
    {
        Part p = c.Prototype as Part;
        if (p == null) continue;
        foreach (Body b in p.Bodies) yield return b;             // corpos dos componentes
    }
}
```

6.5 Medidas: bounding box, volume, faces, raios

Assinatura geométrica do Copilot

```
// bounding box absoluto
double[] box = new double[6];
uf.Modl.AskBoundingBox(body.Tag, box);

// area e volume (props[0] = area, props[1] = volume)
double[] acc = { 0.999 }; double[] props = new double[47]; double[] stats = new double[13];
uf.Modl.AskMassProps3d(new Tag[] { body.Tag }, 1, 1, 4, 1.0, 1, acc, props, stats);

// numero de faces
int faces = body.GetFaces().Length;

// raio de uma face cilindrica (HoleDiameterColorizer)
MeasureManager mm = workPart.MeasureManager; // via theSession.Measurement.GetFaceProperties(...)
```

7. Drafting automático

O journal `CRIAR_DRAFTING_TRES_VISTAS_DINAMICO` cria folha A4, vista base, projetada e dois cortes, com escala e posição calculadas pela peça. Ele condensa três lições de NXOpen.

7.1 Trocar de aplicação e criar a folha

Folha sem template

```
theSession.ApplicationSwitchImmediate("UG_APP_DRAFTING");
workPart.Drafting.EnterDraftingApplication();

DraftingDrawingSheetBuilder sb = workPart.DraftingDrawingSheets.CreateDraftingDrawingSheetBuilder(null);
sb.Option = DrawingSheetBuilder.SheetOption.CustomSize; // sem template = sem bloco de titulo
sb.Height = 210.0; sb.Length = 297.0; // A4 paisagem
sb.Units = DrawingSheetBuilder.SheetUnits.Metric;
sb.ProjectionAngle = DrawingSheetBuilder.SheetProjectionAngle.Third;
DraftingDrawingSheet folha = (DraftingDrawingSheet)sb.Commit();
sb.Destroy();
folha.Open();
```

7.2 O ponto de colocação não é o centro

A lição mais cara do dia: `Placement.Placement.SetValue(null, workView, ponto)` posiciona o **ponto de referência** da vista — a origem do modelo — e não o centro da geometria. Uma peça modelada de 0 a 250 mm "cresce" para fora da folha. A solução é medir depois de criar e recentrar:

LayoutFolha – medir e recentrar

```
public static double[] Bordas(DraftingView vista) // [xmin, ymin, xmax, ymax] na folha
{
    double[] b = new double[4];
    UFSession.GetUFSession().Draw.AskViewBorders(vista.Tag, b);
    return b;
}

public static void MoverParaCentro(DraftingDrawingSheet folha, DraftingView vista, Point3d centroDesejado)
{
    Point3d atual = Centro(vista);
    Point3d refAtual = vista.GetDrawingReferencePoint();
    vista.SetDrawingReferencePoint(new Point3d(
        refAtual.X + (centroDesejado.X - atual.X),
        refAtual.Y + (centroDesejado.Y - atual.Y), 0.0));
    UFSession.GetUFSession().Draw.UpdateOneView(folha.Tag, vista.Tag);
}
```

7.3 Escala que cabe: contar as vistas de verdade

A primeira versão considerava duas vistas na horizontal; a folha tinha três (base + dois cortes). O cálculo certo soma as larguras reais das vistas, as folgas em mm de folha (não divididas pela escala) e um fator de ocupação para deixar respiro:

CalcularDenominadorEscala

```
double pecaHoriz = dx + 2 * Math.Max(dy, dz); // base + 2 cortes (mm de peca)
double pecaVert = dy + dz; // base + frente
double largDisp = (folhaL - 2 * MARGEM) * OCUPACAO_MAX - 2 * FOLGA; // mm de folha
double altDisp = (folhaH - 2 * MARGEM - blocoTitulo) * OCUPACAO_MAX - FOLGA;

foreach (double d in new[] { 1, 2, 2.5, 5, 10, 20, 25, 50, 100 }) // ISO 5455
    if (pecaHoriz / d <= largDisp && pecaVert / d <= altDisp) return d;
```

Para a peça de 250 × 200 × 50 na A4, isso dá 1:5 — e todas as vistas dentro da folha.

8. Windows Forms essencial

Windows Forms é a biblioteca de interface gráfica mais antiga do .NET — e, para uma DLL que roda dentro do NX, a mais confiável: sem runtime extra, sem XAML, sem conflito com o .NET Framework que o NX hospeda. O PATHNC inteiro é WinForms escrito em código, sem o Designer.

8.1 Os tijolos

Controle	Para quê	No PATHNC
Form	A janela	<code>Form1</code> , <code>CopilotResultForm</code> , <code>QuoteResultForm</code>
Panel	Contêiner; agrupa e posiciona	Cada página é um <code>Panel</code> ; cartões do Theme também
Button	Ação	<code>Theme.MakeButton</code>
Label	Texto	Títulos, dicas, status
ComboBox	Lista suspensa	Idioma do Shop Doc, máquinas, estratégias
NumericUpDown	Número com setas	Quantidade, hora-máquina, margem
DataGridView	Tabela	Peças similares, operações, histórico de orçamentos
PictureBox	Imagem	Miniaturas, ícones de fixture
ProgressBar	Progresso	Barra de status do painel
<code>SplitContainer</code> , <code>TableLayoutPanel</code> , <code>FlowLayoutPanel</code>	Layouts	Janela do Copilot, faixa de imagens, navegação

8.2 Posicionar: Location/Size, Dock e Anchor

Há duas filosofias. **Absoluta**: você diz onde cada controle fica (`Location` , `Size`) — é como as páginas do PATHNC funcionam, com uma variável `y` que vai descendo. **Relativa**: o controle se cola a uma borda do pai (`Dock`) ou acompanha o redimensionamento (`Anchor`) — é como o painel principal se monta.

●●● Form1 – os dois estilos de layout

```
// Absoluta: fluxo vertical de uma pagina
int y = 116;
SectionTitle(page, "SETUP SHEET", PAD, y); y += 22;
ActionButton(page, "Generate Setup Sheet", PAD, y, CONTENT_WIDTH, 40, ...); y += 48;

// Relativa: o esqueleto do Form1 (a ORDEM de Controls.Add importa no Dock)
this.Controls.Add(statusBar); // Dock = Bottom -> adicionado antes do Fill
this.Controls.Add(contentPanel); // Dock = Fill -> ocupa o que sobrou
this.Controls.Add(navPanel); // Dock = Left
this.Controls.Add(header); // Dock = Top
```

Ordem do Dock: o WinForms processa `Dock` na ordem inversa de adição. Um `Fill` adicionado antes de um `Bottom` cobre a barra de status. Foi por isso que o comentário no `BuildUi` diz que a status bar precisa entrar antes do `contentPanel` .

8.3 Eventos e lambdas

Um botão avisa que foi clicado pelo evento `Click` . A forma mais direta de reagir é uma lambda:

●●● Eventos: consumir e publicar

```
btn.Click += (s, e) => RunSelectedShopDocumentation(); // s = quem disparou, e = argumentos

// lambda que captura variavel do loop: precisa de copia local
for (int i = 0; i < navLabels.Length; i++)
{
    int capturedIndex = i; // sem isso, todos os botoes usariam o ultimo i
    btn.Click += (s, e) => SelectPage(capturedIndex);
}

// evento proprio, para outras janelas se inscreverem
public event Action<string, int, bool> ProgressChanged;
if (ProgressChanged != null) ProgressChanged(status, percent, false);
```

8.4 Modal ou modeless

	<code>ShowDialog(this)</code> — modal	<code>Show(this)</code> — modeless
Bloqueia o chamador?	Sim, até fechar	Não
Devolve resultado?	<code>DialogResult</code>	Não — use eventos
Uso no PATHNC	<code>ActualTimeDialog</code> , <code>OpenFileDialog</code> , <code>ToolDatabaseManagerForm</code>	<code>Form1</code> , <code>ChatAssistantForm</code> , <code>CopilotResultForm</code> , <code>QuoteResultForm</code>
Exige	—	<code>GetUnloadOption = Explicitly</code> e guardar a referência para não abrir duas

Uma instância só de uma janela modeless

```
private CopilotResultForm _copilotResultForm;
if (_copilotResultForm == null || _copilotResultForm.IsDisposed)
{
    _copilotResultForm = new CopilotResultForm();
    _copilotResultForm.ApplyRequested += ApplyCopilotStrategy; // a janela pede, o Form1 executa
    _copilotResultForm.Show(this);
}
_copilotResultForm.Carregar(atual, sug);
_copilotResultForm.Activate();
```

8.5 A thread de UI e o Application.DoEvents()

Tudo — o NX, o painel, o NXOpen — roda na mesma thread. Enquanto uma automação de 40 segundos roda, a janela não repinta e a barra de progresso não anda. A solução do PATHNC é chamar `Application.DoEvents()` depois de cada atualização de status, forçando o processamento de mensagens pendentes:

Por que o DoEvents existe

```
public void ReportProgress(string status, int percent)
{
    lblStatusBar.Text = status;
    progressBar.Value = Math.Max(0, Math.Min(100, percent));
    Application.DoEvents(); // deixa a janela repintar antes de continuar
}
```

Não use threads com NXOpen. A API do NX não é thread-safe; chamar `Session` de um `Task.Run` trava ou corrompe. `DoEvents` é feio, mas é a ferramenta certa aqui. Para trabalhos longos sem NX (chamada HTTP ao Claude, consulta SQL grande) aí sim `async/await` é seguro.

8.6 Designer ou código?

O Designer gera `Form1.Designer.cs` com `InitializeComponent()`. O PATHNC mantém esse arquivo mínimo e constrói tudo em `BuildUi()`. Razões: código é versionável e legível em diff; o Designer quebra com controles criados em loop; e helpers como `ActionButton` garantem consistência — o Theme só existe porque a construção era programática.

9. Anatomia do painel PATHNC

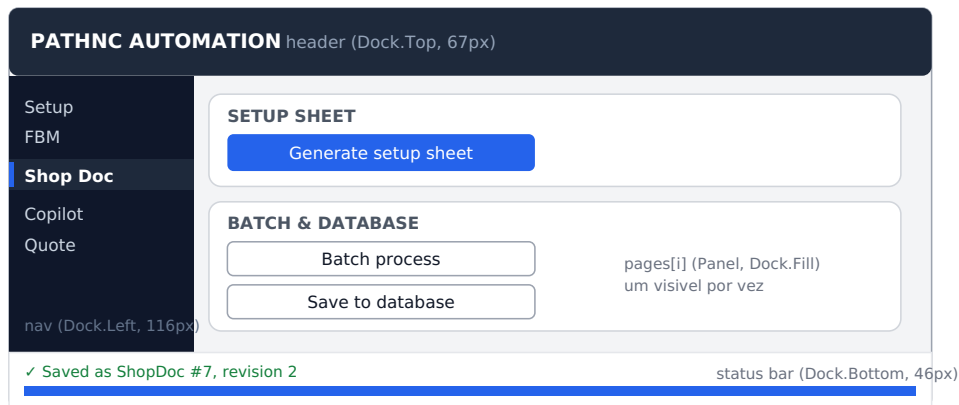


Figura 3 — As quatro regiões do Form1 e como se encaixam por Dock.

9.1 Navegação por índice

Dois arrays paralelos definem o menu; um terceiro, as páginas. Adicionar uma página é acrescentar uma linha em cada um — foi assim que Copilot e Quote entraram:

```
BuildUi / SelectPage

string[] navLabels = { "⦿ Setup", "○ FBM", /* ... */ "⦿ Shop Doc", "⦿ Copilot", "$ Quote" };
string[] navFullNames = { "Setup", "Feature Based Machining", /* ... */ };
bool[] navIsNew = new bool[navLabels.Length];
navIsNew[navLabels.Length - 1] = true; // selo NEW sempre no ultimo

pages = new Panel[navLabels.Length];
pages[0] = BuildPageSetup();
/* ... */
pages[12] = BuildPageCopilot();
pages[13] = BuildPageQuote();

private void SelectPage(int index)
{
    for (int i = 0; i < pages.Length; i++)
    {
        bool active = (i == index);
        pages[i].Visible = active; // so uma pagina visivel
        navButtons[i].Tag = active; // o Paint desenha a faixa azul se Tag == true
        navButtons[i].Invalidate();
    }
}
```

9.2 Helpers: a razão de as páginas serem curtas

Cada página é uma sequência de chamadas a cinco helpers. Eles concentram estilo e espaçamento; mudar um helper muda 14 páginas. Com o Theme, o helper virou um tradutor de intenção:

```
ActionButton depois do Theme

// A cor passada pelas paginas vira TIPO de botao:
// ColorPrimary -> primario (azul, unico por pagina)
// ColorDanger -> perigoso (borda vermelha)
// demais -> secundario (branco com borda)
private Button ActionButton(Panel page, string text, int x, int y, int w, int h,
    Color back, Color hover, EventHandler onClick)
{
    Theme.ButtonKind kind = Theme.ButtonKind.Secondary;
    if (back == ColorPrimary) kind = Theme.ButtonKind.Primary;
    else if (back == ColorDanger) kind = Theme.ButtonKind.Danger;
    int height = kind == Theme.ButtonKind.Primary ? Theme.PrimaryButtonHeight : Theme.ButtonHeight;
    Button btn = Theme.MakeButton(kind, text, x, y + (h - height) / 2, w, height, onClick);
    page.Controls.Add(btn);
    return btn;
}
```

O par `SectionTitle / Divider` ganhou um segundo papel: abrir e fechar um cartão. O cartão é um `Panel` branco com borda, criado *depois* dos controles e enviado para trás com `SendToBack()` — assim os controles ficam por cima sem precisar de

reparent.

9.3 Feedback: progresso e status

Método	Quando	Efeito
ReportProgressIndeterminate(msg)	Etapa "caixa-preta" (classe que não reporta progresso)	Pula para 30% e mostra a mensagem; nunca anda para trás
ReportProgress(msg, pct)	Progresso conhecido (grupo 3/8)	Barra contínua; 100% fica verde
ShowSuccess(msg) / ShowWarning(msg)	Resultado sem interromper	Texto verde/laranja por alguns segundos (Timer)
MessageBox.Show(...)	Erro ou confirmação	Modal — só quando o usuário precisa decidir

9.4 O padrão "steps" para ciclos longos

●●● RunAutoDrillCycle – array de Actions com rótulos

```
string[] stepLabels = { "Recognizing features...", "Creating feature groups...", "Drilling, counterboring and threading..." };
Action[] steps      = { () => RECOGNIZE_FEATURES.Run(null),
                        () => CREATE_GROUP_FEATURES.Run(null),
                        () => AUTODRILL_GEOMETRIA_PROPRIA.Run(null) };

try
{
    for (int i = 0; i < steps.Length; i++)
    {
        ReportProgressIndeterminate(stepLabels[i]);
        steps[i]();
    }
    ReportProgress("Auto Drill completed.", 100);
}
catch (Exception ex)
{
    ReportProgress("Error in Auto Drill.", 0);
    MessageBox.Show("Error in Auto Drill:\n" + ex.Message, "Auto Drill", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

O mesmo método serve ao botão e ao gatilho de texto do AI Assistant. Uma única fonte de verdade para a sequência.

10. Criando uma página nova, passo a passo

Vamos reconstruir a página **Quote** como ela foi feita — é o roteiro para qualquer página futura.

Passo 1 — Campos da classe

Controles que outros métodos precisam ler viram campos da `partial class Form1`:

```
Campos

private NumericUpDown nudQuoteQty;
private NumericUpDown nudQuoteMachineRate;
private NumericUpDown nudQuoteMargin;
private Label lblQuoteStatus;
private bool _quoteSettingsLoaded;
```

Passo 2 — O método Build

```
BuildPageQuote (resumido)

private Panel BuildPageQuote()
{
    Panel page = new Panel { Dock = DockStyle.Fill, BackColor = ColorContentBg, AutoScroll = true };
    PageHeader(page, "Quote", "Cost and lead time estimate for the current part");
    int y = 116;

    SectionTitle(page, "BATCH", PAD, y); y += 22;
    FieldLabel(page, "QUANTITY", PAD, y); y += 14;
    nudQuoteQty = Nud(page, y, 1, 10000, 1, 0); y += 36;
    Divider(page, PAD, y, CONTENT_WIDTH); y += 16;

    SectionTitle(page, "ESTIMATE", PAD, y); y += 22;
    ActionButton(page, "Estimate Current Part", PAD, y, CONTENT_WIDTH, 44, ColorPrimary, ColorPrimaryHover,
        (s, e) => RunQuoteEstimate()); y += 52;
    lblQuoteStatus = new Label { Location = new Point(PAD, y), Size = new Size(CONTENT_WIDTH, 40),
        Font = new Font("Segoe UI", 7.5F), ForeColor = ColorTextMuted };
    page.Controls.Add(lblQuoteStatus); y += 46;

    // carrega parametros do banco na primeira vez que a pagina aparece
    page.VisibleChanged += (s, e) => { if (page.Visible && !_quoteSettingsLoaded) LoadQuoteSettings(); };
    return page;
}
```

Repare no `VisibleChanged`: consultar o banco no `BuildUi` atrasaria a abertura do painel e falharia se o SQL estivesse fora; carregar sob demanda resolve os dois.

Passo 3 — Um helper novo quando o padrão se repete

A página tem sete `NumericUpDown` iguais. Sete blocos de inicialização viraram um helper:

```
Helper Nud

private NumericUpDown Nud(Panel page, int y, decimal min, decimal max, decimal val, int decimals)
{
    NumericUpDown n = new NumericUpDown
    {
        Location = new Point(PAD, y), Size = new Size(CONTENT_WIDTH, 23),
        Minimum = min, Maximum = max, Value = val, DecimalPlaces = decimals,
        Font = new Font("Segoe UI", 9F), ThousandsSeparator = true,
    };
    page.Controls.Add(n);
    return n;
}
```

Passo 4 — O handler: UI → serviço → UI

O handler não sabe calcular orçamento. Ele lê a UI, chama o serviço e mostra o resultado. Essa separação é o que permite testar o `QuoteService` sem abrir o NX (capítulo 14).

RunQuoteEstimate – o formato de todo handler do PATHNC

```
private void RunQuoteEstimate()
{
    Cursor previous = Cursor.Current;
    try
    {
        Cursor.Current = Cursors.WaitCursor;
        ReportProgressIndeterminate("Quote: reading current part and searching history...");

        ShopDocModel atual = StrategyHistoryService.AssinaturaAtual(); // NX
        StrategySuggestion sug = new StrategyHistoryService().Sugerir(atual, top, minScore); // SQL
        QuoteEstimate q = new QuoteService().Estimate(atual, sug, (int)nudQuoteQty.Value,
                                                       ReadQuoteSettingsFromUi()); // calculo

        ReportProgress(string.Format("Quote: {0} {1:N2}, {2:F0} days.", q.Settings.Currency, q.TotalPrice, q.LeadTimeDays),
100);
        new QuoteResultForm(q, atual).Show(this); // UI
    }
    catch (Exception ex)
    {
        ReportProgress("Quote: error.", 0);
        MessageBox.Show("Estimate failed:\n" + ex.Message, "Quote", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally { Cursor.Current = previous; }
}
```

Passo 5 — Registrar na navegação

Um item em `navLabels`, um em `navFullNames`, uma linha `pages[13] = BuildPageQuote();`. Pronto.

Checklist de uma página: (1) uma ação primária no máximo; (2) parâmetros que persistem vêm do banco sob demanda; (3) handler com try/catch/finally, cursor de espera e status; (4) lógica em serviço, nunca no handler; (5) resultado grande em janela separada, não no painel estreito.

11. Janelas secundárias, grids e imagens

11.1 Uma janela construída à mão

As janelas do Copilot e do Quote não têm Designer: herdam de `Form` e montam tudo no construtor. O padrão é o mesmo do painel — cabeçalho escuro, corpo, rodapé com ações.

```
QuoteHistoryForm - esqueleto

public class QuoteHistoryForm : Form
{
    private readonly TextBox _txtFilter = new TextBox();
    private readonly DataGridView _grid = new DataGridView();
    private List<QuoteRecord> _rows = new List<QuoteRecord>();

    public QuoteHistoryForm()
    {
        Text = "PATHNC - Quote History";
        Size = new Size(1000, 560);
        StartPosition = FormStartPosition.CenterScreen;
        Theme.StyleForm(this);

        Panel top = new Panel { Dock = DockStyle.Top, Height = 44 };
        /* filtro + botoes */
        Theme.StyleGrid(_grid);
        _grid.Dock = DockStyle.Fill;

        Controls.Add(_grid);
        Controls.Add(top);

        _grid.CellDoubleClick += (s, e) => ShowDetail();
        _txtFilter.KeyDown += (s, e) => { if (e.KeyCode == Keys.Enter) { Reload(); e.SuppressKeyPress = true; } };
        Reload();
    }
}
```

11.2 DataGridView com tipos anônimos

A forma mais rápida de mostrar uma lista: projetar os objetos em tipos anônimos com os nomes de coluna que você quer ver. O grid gera as colunas sozinho.

```
Binding rápido

_grid.DataSource = _rows.ConvertAll(q => new
{
    Id      = q.QuoteId,
    Date    = q.CreatedAt.ToLocalTime().ToString("dd/MM/yyyy HH:mm"),
    Part    = q.PartName,
    Customer = q.CustomerName,
    Qty     = q.Quantity,
    Total   = q.Currency + " " + q.TotalPrice.ToString("N2"), // formatado como texto
    Days    = q.LeadTimeDays.ToString("F0"),
});

// e para saber qual objeto real esta na linha selecionada:
int i = _grid.SelectedRows[0].Index;
QuoteRecord q = _rows[i]; // mesma ordem da lista projetada
```

Limitação: colunas de tipo anônimo são texto; para ordenar numericamente, use uma classe própria com propriedades numéricas ou um `DataTable`.

11.3 Grid mestre-detache

●●● CopilotResultForm – seleção no grid 1 alimenta o grid 2

```
_gridParts.SelectionChanged += (s, e) =>
{
    ShowOperationsOfSelected(); // popula o segundo grid
    _btnApply.Enabled = VizinhaSelecionada != null && VizinhaSelecionada.Operations.Count > 0;
};

private void ShowOperationsOfSelected()
{
    if (_sug == null || _gridParts.SelectedRows.Count == 0) return;
    int idx = _gridParts.SelectedRows[0].Index;
    SetImage(_picVizinha, _sug.Vizinhos[idx].Thumbnail);
    _gridOps.DataSource = _sug.Vizinhos[idx].Operations.Select(o => new { o.Seq, o.OperationName, o.ToolName, o.Rpm }).ToList();
}
```

11.4 Imagens a partir de bytes

●●● PictureBox e Dispose

```
private static void SetImage(PictureBox p, byte[] png)
{
    Image old = p.Image;
    p.Image = (png == null || png.Length == 0) ? null : Image.FromStream(new MemoryStream(png));
    if (old != null) old.Dispose(); // Image e recurso nao gerenciado: libere sempre
}
p.SizeMode = PictureBoxSizeMode.Zoom; // mantem proporcao dentro da caixa
```

11.5 Diálogo com validação

Um `ShowDialog` devolve `DialogResult.OK` quando o botão com `DialogResult = OK` é clicado. Para não fechar quando a validação falha, o handler zera o resultado:

●●● ActualTimeDialog – DialogResult.None

```
ok.Click += (s, e) =>
{
    if (_cmbPart.SelectedItem == null || _nudTime.Value <= 0)
    {
        MessageBox.Show("Select a part and enter the actual machining time.", "Actual time");
        DialogResult = DialogResult.None; // cancela o fechamento
        return;
    }
    new QuoteService().RegisterActualTime(_cmbPart.SelectedItem.ToString(), (double)_nudTime.Value, ...);
};
```

11.6 Comunicação entre janelas: eventos, não referências

A janela do Copilot não chama o NX; ela dispara `ApplyRequested(vizinha)`. Quem escuta é o `Form1`, que tem a barra de progresso e o contexto. Isso mantém a janela reutilizável e testável, e evita que duas janelas precisem se conhecer.

12. C# que o projeto usa

Não é um curso de C#; é o subconjunto que aparece no PATHNC, explicado pelo uso. O projeto está em **C# 7.3** (limite do .NET Framework 4.8), por isso você verá `Tuple.Create` em vez de tuplas nomeadas e nenhum `record`.

12.1 Classe, objeto, campo, propriedade

Modelo de dados

```
public class ShopDocModel // classe: o molde
{
    public string PartName; // campo publico (simples, sem validacao)
    public double? SizeX; // nullable: pode nao ter valor
    public List<ShopDocOperationModel> Operations = new List<ShopDocOperationModel>();

    public double TotalTimeMin // propriedade calculada (so get)
    {
        get { return Operations.Sum(o => o.TimeMin); }
    }
}
ShopDocModel doc = new ShopDocModel { PartName = "PLACA_P1" }; // objeto + inicializador
```

12.2 static: sem instância

`static` pertence à classe, não ao objeto. Todo journal do PATHNC é uma classe com `public static void Run(string[] args)` — chamável sem `new`. Helpers puros (`Theme`, `LayoutFolha`) são classes estáticas inteiras.

Classe estática

```
public static class Theme
{
    public static readonly Color Accent = Color.FromArgb(37, 99, 235); // constante de leitura
    public const int ButtonHeight = 30; // constante de compilacao
    public static Button PrimaryButton(string text, int x, int y, int w, EventHandler onClick) { ... }
}
Button b = Theme.PrimaryButton("Save", 16, 40, 232, (s, e) => Save());
```

12.3 Interface e herança

Interface é um contrato: "quem implementar isto tem estes membros". **Herança** é reaproveitar implementação. O PATHNC usa as duas:

Interface x herança

```
// Contrato: toda estrategia de usinagem tem um nome e sabe se executar
public interface IMachiningStrategy
{
    string FeatureName { get; }
    void Execute();
}
public class M8TapStrategy : IMachiningStrategy
{
    public string FeatureName { get { return "M8 Tap"; } }
    public void Execute() { M8.Run(null); }
}

// Heranca: todas as janelas SAO um Form, e ganham de graca Show, Close, Controls...
public class CopilotResultForm : Form { ... }
```

	Interface	Herança de classe
Quantas por classe	Várias	Uma só
Traz código?	Não (só assinaturas)	Sim
Use quando	Coisas diferentes fazem a mesma ação (estratégias, repositórios, testes com mock)	Coisa é um caso especial de outra (Form, Exception)

12.4 Genéricos

Um método genérico recebe o tipo como parâmetro. `FindGroupByNameAndType<T>` serve para `Method`, `FeatureGeometry` ou `NCGroup` com uma implementação só. `where T : class` permite o `as T`.

12.5 IEnumerable, yield e LINQ

● ● ● StrategyHistoryService – LINQ como SQL em memória

```
// yield: produz itens sob demanda sem montar uma lista
private static IEnumerable<NXOpen.CAM.Tool> GetAllTools(Part workPart)
{
    foreach (CAMObject obj in workPart.CAMSetup.CAMGroupCollection.ToArray())
    {
        NXOpen.CAM.Tool tool = obj as NXOpen.CAM.Tool;
        if (tool != null) yield return tool;
    }
}

// LINQ: consultas em memória com a mesma cabeça do SQL
List<ToolParamSuggestion> parametros = vizinhos
    .SelectMany(v => v.Operations) // achata: todas as operacoes de todas as vizinhas
    .Where(o => !string.IsNullOrEmpty(o.ToolName)) // WHERE
    .GroupBy(o => o.ToolName) // GROUP BY
    .Select(g => new ToolParamSuggestion
    {
        ToolName = g.Key,
        RpmMediana = Mediana(g.Where(o => o.Rpm > 0).Select(o => o.Rpm.Value)),
        Ocorrencias = g.Count()
    })
    .OrderByDescending(p => p.Ocorrencias) // ORDER BY
    .ToList();
```

12.6 Nullable, null e o operador ??

● ● ● Trabalhando com ausência de valor

```
double? rpm = LerDouble(uf, op.Tag, UFConstants.UF_PARAM_SPINDLE_RPM); // pode ser null
if (rpm.HasValue && rpm.Value > 0) { ... }
cmd.Parameters.AddWithValue("@Rpm", (object)rpm ?? DBNull.Value); // null -> NULL do SQL
string nome = doc.HtmlFileName ?? "(none found)"; // valor padrao
```

12.7 Exceções: quem trata o quê

- **Serviços** (`StrategyApplier` , `ShopDocRepository`) desfazem o que fizeram (undo mark, rollback) e **relançam** (`throw` ;).
- **Handlers** de UI capturam `Exception` , mostram `MessageBox` e restauram cursor/status no `finally` .
- **Leituras opcionais** (miniatura, HTML) engolem a exceção e devolvem `null` : a ausência não pode impedir o save.
- `NXException` tem `ErrorCode` ; `InvalidOperationException` é a escolha para erros de lógica sua (não use `new NXException` — não tem construtor público).

13. Padrões de projeto no PATHNC

Padrões de projeto são nomes para soluções que se repetem. O PATHNC não foi escrito "para usar padrões" — eles apareceram porque resolviam problemas concretos. Aqui está cada um, com o problema que resolveu.

13.1 Strategy — muitas formas de fazer a mesma coisa

Problema: dezenas de usinagens (M6 a M20, soquetes, furos) selecionáveis num combo, sem um `switch` gigante.

```
IMachiningStrategy + MachiningContext

public class MachiningContext
{
    private readonly Dictionary<string, IMachiningStrategy> _map = new Dictionary<string, IMachiningStrategy>();
    public void Register(IMachiningStrategy s) { _map[s.FeatureName] = s; }
    public void Execute(string featureName) { _map[featureName].Execute(); }
}

// Form1_Load: registra todas
context.Register(new M6TapStrategy()); context.Register(new M8TapStrategy()); /* ... */
cmbMetricTap.DataSource = strategiesMetricTap; // o combo lista objetos IMachiningStrategy
cmbMetricTap.DisplayMember = "FeatureName";

// botao Run: nao sabe qual estrategia e - so pede para executar
if (cmbMetricTap.SelectedItem is IMachiningStrategy strategy)
    context.Execute(strategy.FeatureName);
```

Adicionar uma rosca nova é criar uma classe e registrar. O botão não muda.

13.2 Builder — o padrão do próprio NX

Você já usa: `CreateCavityMillingBuilder` → propriedades → `Commit` → `Destroy`. O NX separa a configuração (muitos passos) da criação (um passo atômico). O `DraftingDrawingSheetBuilder`, o `ImageExportBuilder` e todos os outros seguem o mesmo contrato.

13.3 Repository — o banco atrás de uma porta

Problema: SQL espalhado pelos handlers de UI. **Solução:** uma classe por "coisa" persistida, com métodos que falam a língua do domínio:

```
Repository / Service

public class ShopDocRepository
{
    public Tuple<int,int> Save(ShopDocModel doc) { /* transacao, INSERTs */ }
}

public class QuoteService
{
    public QuoteEstimate Estimate(ShopDocModel atual, StrategySuggestion sug, int qty, QuoteSettings s) { ... }
    public int SaveQuote(QuoteEstimate q, ...) { ... }
    public List<QuoteRecord> ListQuotes(string filtro = null) { ... }
    public bool RegisterActualTime(string partName, double min, double? setupMin, string note) { ... }
}
```

O handler chama `new ShopDocRepository().Save(doc)` e não sabe se por trás há SQL Server, SQLite ou um arquivo. Trocar o banco é trocar uma classe.

13.4 Facade — um método na frente de vários

`RunAutoDrillCycle()` esconde três classes e a barra de progresso atrás de uma chamada. Botão e chat usam a mesma fachada. Mesmo papel: `THREE_PLUS_TWO_PIPELINE` (encadeia Rough → Rest Mill → Semi-Finish).

13.5 Template Method — mesma estrutura, detalhes diferentes

As oito classes `SHOP_DOCUMENTATION_PT/EN/DE/...` fazem o mesmo (varrer o setup, capturar imagens, montar HTML) mudando os textos. Hoje são cópias; o padrão sugere uma classe base com o algoritmo e subclasses (ou um dicionário de idioma) fornecendo só as strings. É uma refatoração pendente que reduziria oito arquivos a um.

13.6 Observer — eventos

`ProgressChanged` no `Form1` e `ApplyRequested` no `CopilotResultForm`: quem produz não conhece quem consome. O `ChatAssistantForm` se inscreve para mostrar a barra de progresso na própria janela.

13.7 Singleton — uma instância só

`Session.GetSession()` e `UI.GetUI()` são singletons do NX. O `Form1` aplica a ideia às janelas modeless (uma instância de `_copilotResultForm`, reaproveitada).

13.8 Factory — criar sem expor o como

`Theme.MakeButton(kind, ...)` decide cor, fonte, borda e cantos a partir de um enum. As páginas pedem "um botão primário" e recebem o objeto pronto. Se amanhã o primário for verde, muda num lugar.

13.9 Separação em camadas

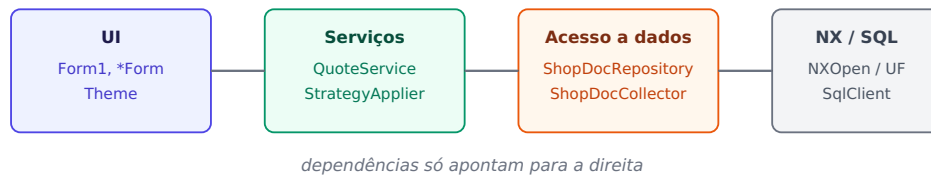


Figura 4 — Camadas do PATHNC. A UI nunca fala SQL; o repositório nunca abre MessageBox.

14. Organização, testes e deploy

14.1 Convenções que o projeto segue

Item	Convenção	Exemplo
Journal/automação executável	Classe MAIÚSCULA com <code>Run(string[])</code> , um arquivo, uma pasta por família	<code>FBM_RECTANGULAR_POCKET_ROUGH.Run(null)</code>
Serviço/repositório	PascalCase, sufixo Service/Repository, namespace <code>PathNCAutomation.*</code>	<code>PathNCAutomation.ShopDoc.QuoteService</code>
Janela	Sufixo Form/Dialog, namespace do form principal	<code>nxteste2.CopilotResultForm</code>
Campos privados	<code>_camelCase</code>	<code>_copilotResultForm</code> , <code>_cs</code>
Constantes de configuração	<code>const</code> no topo da classe, com comentário	<code>OPER_NAME</code> , <code>TEMPLATE_A4</code> , <code>TemplateMap</code>
Comentários	Explicam o <i>porquê</i> (bug corrigido, decisão), não o <i>o quê</i>	"FIX: antes a regra era criada com <code>workPart.ScRuleFactory...</code> "

14.2 Testar sem o NX

O NXOpen não roda fora do NX — então código que depende dele não é testável em xUnit. A saída é isolar a dependência atrás de uma interface e testar a lógica pura:

0 que dá para testar

```
// A logica de similaridade nao toca o NX: testavel
public static double CalcularScore(ShopDocModel a, SimilarPart b) { ... }

[Fact]
public void PecasIguaisTemScoreUm()
{
    var a = new ShopDocModel { SizeX = 250, SizeY = 200, SizeZ = 50, VolumeMm3 = 2.5e6, FaceCount = 38 };
    var b = new SimilarPart { SizeX = 50, SizeY = 250, SizeZ = 200, VolumeMm3 = 2.5e6, FaceCount = 38 }; // rotacionada
    Assert.Equal(1.0, StrategyHistoryService.CalcularScore(a, b), 3);
}

// A coleta do NX fica atras de uma interface; no teste, um fake devolve um modelo pronto
public interface IPartSignatureSource { ShopDocModel AssinaturaAtual(); }
```

Regra: quanto mais lógica sair dos handlers e das classes que chamam `Session.GetSession()`, mais o projeto é testável. `QuoteService.Estimate` já é puro; `StrategyApplier.Apply` não — e está tudo bem, desde que a parte que não é NX (o `TemplateMap`, a geração de nomes únicos) esteja separada.

14.3 Versionamento

- Git com `.gitignore` para `bin/`, `obj/`, `*.user` e o arquivo `.connection` (contém nome de servidor).
- Uma mensagem de commit por funcionalidade: "Shop Doc: save to database with revisioning".
- Scripts SQL numerados no repositório, ao lado do código que os exige.

14.4 Deploy numa máquina de cliente

1. Compilar em **Release x64**.
2. Copiar a pasta `bin\Release` inteira (DLL, `Resources\*.dtx`, imagens).
3. Criar `PathNCAutomationDB.connection` com o servidor do cliente.
4. Rodar os scripts SQL na ordem (Schema, `Migracao_1..4`).
5. No NX: Tools → Journal → Play → a DLL. Para virar botão: `UGII_USER_DIR` com um `.men` apontando para `Main`.
6. Licença: NX Open Author para *desenvolver*; para *executar* a DLL, a licença de NX do usuário basta (verifique o bundle do cliente).

Assinatura de DLL não é obrigatória para o NX carregar, mas evita o aviso de "origem desconhecida" em ambientes com política de execução restritiva. Um certificado de code signing resolve; para ferramentaria pequena, raramente é necessário.

15. Compêndio de erros e soluções

Sintoma	Causa	Solução
Unable to load DLL 'libugini'	Executou fora do NX	Tools → Journal → Play; nunca F5 no VS
No Main found	Journal aponta para arquivo sem Main (classe auxiliar)	Rodar o arquivo com Main , ou juntar num só
Build: file in use	DLL carregada com Explicitly	File → Utilities → Unload Shared Image
Botões do painel param de responder	GetUnloadOption = Immediately com form modeless	Devolver Explicitly
Seleção não pega nada	Máscara errada (UF_edge_type , subtipo + flag juntos)	(UF_solid_type, 0, UF_UI_SEL_FEATURE_ANY_EDGE)
Seleção pega o componente inteiro	Sem máscara ou escopo Within Work Part	Máscara específica; escopo Entire Assembly
Face Collector não seleciona em assembly	Escopo do bloco + regra criada na Work Part errada	Scope no .dlx/SelectObject + ScRuleFactory do OwningPart
Vista de drafting fora da folha	SetValue posiciona a origem do modelo, não o centro	AskViewBorders + SetDrawingReferencePoint
'Path' is ambiguous	NXOpen.CAM.Path x System.IO.Path	Qualificar System.IO.Path
NXException has no constructor	Não é instanciável	InvalidOperationException
Cannot convert NCGroup to Tool	CreateTool devolve base	Cast explícito
UF_PARAM_FEED_CUT_VALUE não existe	Nome errado	UF_PARAM_FEED_CUT
BackgroundOptions.CurrentDisplay não existe	Enum varia por versão	Original , ou omitir
Ambiguity between CopilotResultForm.X (dezenas)	Mesmo arquivo em duas pastas do projeto	Excluir a cópia
Load hides inherited member Form.Load	Método com nome de evento herdado	Renomear (Carregar)
App.config não é lido	Executável é o ugraf.exe	Arquivo ao lado da DLL / variável de ambiente
Login failed no SQL	Banco em outra instância	.connection com .\SQLEXPRESS
Invalid column name	Tabela antiga; script pulou criação	Migração com COL_LENGTH
Painel some atrás do NX	Janela não é filha do NX	TopMost = true ; Session.Pause() / SetForegroundWindow para foco
Barra de progresso não anda	Thread de UI ocupada pelo NXOpen	Application.DoEvents() após cada atualização

16. Referências

NXOpen

- Siemens — *NX Open Programmer's Guide* e *NX Open .NET Reference Guide* (na pasta UGD0C da instalação ou no Support Center).
- Siemens — *Block UI Styler* e *Open C Reference* (funções UF_*).
- Siemens GTAC — exemplos de código por área (CAM, Drafting, Assemblies).
- nxjournaling.com — a maior coleção comunitária de journals comentados; foi de lá a técnica de extrair o bloco de título da coleção da peça.
- Fórum Siemens Community — NX Programming & Customization.

C# e Windows Forms

- Microsoft Learn — *Windows Forms documentation* (.NET Framework): Dock/Anchor, DataGridView, threading model.
- Microsoft Learn — *C# language reference* (o projeto está em C# 7.3).
- Jon Skeet, *C# in Depth* — genéricos, LINQ, lambdas com a profundidade certa.
- Robert C. Martin, *Clean Code* — os nomes e o tamanho dos métodos do PATHNC seguem daqui.

Padrões de projeto

- Gamma, Helm, Johnson, Vlissides — *Design Patterns* (GoF): Strategy, Builder, Facade, Template Method, Observer, Singleton, Factory.
- Martin Fowler, *Patterns of Enterprise Application Architecture* — Repository e Service Layer.
- refactoring.guru — explicações visuais de cada padrão, com C#.

Deste projeto

- *SQL Server na prática — do NX ao banco de dados* (CADCAMWise, 2026) — o volume irmão, sobre a camada de dados.
- Arquivos citados: Form1.cs , Theme.cs , ROUGHINTERACTIVE.cs , SELECIONAR_TRIM_BOUNDARY.cs , CRIAR_DRAFTING_TRES_VISTAS_DINAMICO.cs , ShopDocDatabase.cs , StrategyHistoryService.cs , StrategyApplier.cs , QuoteService.cs , QuoteForms.cs , CopilotResultForm.cs .

Escrito a partir do código e das decisões do PATHNC AUTOMATION (CADCAMWise) durante o desenvolvimento dos módulos Shop Documentation, Copilot, Quote e do redesign visual, em setembro de 2026. Os trechos foram encurtados para caber na página.